

Otulia Web: The Complete Developer Guide

Table of Contents

- [Chapter 1: Introduction & Architecture](#)
- [Chapter 2: The Data Layer \(MongoDB & Models\)](#)
- [Chapter 3: Backend Services & API \(Node.js & Express\)](#)
- [Chapter 4: Authentication & Authorization](#)
- [Chapter 5: Frontend Architecture \(React & Vite\)](#)
- [Chapter 6: Asset Management & Cloudinary](#)
- [Chapter 7: Search Engine Optimization \(SEO\)](#)

Chapter 1: Introduction & Architecture

1.1 Project Overview

Otulia Web is a high-end luxury asset marketplace. It is built as a full-stack monorepo, separating logic into `client` (React) and `server` (Node.js/Express).

1.2 System Architecture

The system follows a classic **Client-Server-Database** pattern:

- Client:** A Single Page Application (SPA) that handles all UI rendering.
- Server:** A RESTful API that handles business logic and security.
- Database:** MongoDB for flexible, document-based storage of complex assets.

Chapter 2: The Data Layer (MongoDB & Models)

We use **Mongoose** to define the shape of our data.

2.1 The User Model (`User.model.js`)

This model tracks user accounts, permissions, and history.

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true }, // Name of the user, whitespace
  trimmed
  email: { type: String, required: true, unique: true, lowercase: true }, // Primary key,
  always lowercase
  role: { type: String, enum: ["user", "agent", "admin"], default: "user" }, // Role-based
  access control
  plan: { type: String, enum: ["Freemium", "Premium Basic", "Business VIP"], default:
  "Freemium" }, // Tiered features
  isVerified: { type: Boolean, default: false }, // Verification status for dealers
  favorites: [{
    assetId: { type: mongoose.Schema.Types.ObjectId, refPath: "favorites.assetModel" }, //
    Polymorphic ID
    assetModel: { type: String, enum: ["Listing", "CarAsset", "EstateAsset", "YachtAsset",
    "BikeAsset"] } // Tells Mongoose which collection to look in
```

```
  ]]  
}, { timestamps: true }); // Automatically adds 'createdAt' and 'updatedAt'
```

2.2 The Car Asset Model (CarAsset.model.js)

Luxury assets require extremely granular data.

```
const carAssetSchema = new mongoose.Schema({  
  title: { type: String, required: true }, // Display name  
  price: { type: Number, required: true }, // Raw numeric price for sorting/filtering  
  specification: {  
    power: String, // e.g., "800 HP"  
    transmission: String, // e.g., "Automatic"  
    matchingNumbers: String, // Important for luxury/vintage car value  
    accidentFree: String // Crucial for buyer trust  
  },  
  agent: {  
    id: String, // Links to the User who posted it  
    companyLogo: String // Denormalized for faster rendering  
  },  
  status: { type: String, enum: ['Active', 'Sold', 'Rented', 'Draft'], default: 'Active' }  
}, { timestamps: true });
```

Chapter 3: Backend Services & API (Node.js & Express)

3.1 Server Entry Point (server/index.js)

This is the heart of the backend. It boots the database and plugs in all routes.

```
// Initialization  
const app = express();  
connectDB(); // Establishes connection to MongoDB  
  
// Global Middleware  
app.use(express.json()); // Parses incoming JSON payloads  
app.use(corsMiddleware); // Enables Cross-Origin Resource Sharing  
  
// High-Priority Route (SEO)  
app.use("/", sitemapRoutes); // Sitemap must be handled before static files  
  
// API Route Registration  
app.use("/api/auth", authRoutes); // Authentication (Login/Register)  
app.use("/api/assets", assetsRoutes); // Data retrieval for cars, yachts, etc.  
app.use("/api/inventory", inventoryRoutes); // Dealer management tools  
  
// Frontend Serving (Production)  
app.use(express.static(path.join(__dirname, "../client/dist"))); // Serves compiled React files  
app.use((req, res) => {  
  res.sendFile(path.join(__dirname, "../client/dist/index.html")); // Catch-all for React
```

```
Router
});
```

Chapter 4: Authentication & Authorization

4.1 Login Logic (auth.routes.js)

We use JWT (JSON Web Tokens) for secure, stateless sessions.

```
router.post("/login", async (req, res) => {
  const { email, password } = req.body;

  // 1. Find user in database
  const user = await User.findOne({ email });
  if (!user) return res.status(401).json({ error: "INVALID_CREDENTIALS" });

  // 2. Verify hashed password
  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return res.status(401).json({ error: "INVALID_CREDENTIALS" });

  // 3. Generate secure token
  const token = jwt.sign(
    { id: user._id, role: user.role, email: user.email }, // Payload
    process.env.JWT_SECRET, // Secret key from .env
    { expiresIn: "7d" } // Session lasts 1 week
  );

  res.json({ token, user }); // Send back to client
});
```

Chapter 5: Frontend Architecture (React & Vite)

5.1 Global State Management (AuthContext.jsx)

This context makes user data available to every component in the app.

```
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null); // Current logged in user
  const [token, setToken] = useState(localStorage.getItem('token')); // JWT

  const login = async (email, password) => {
    const response = await fetch('/api/auth/login', { /* ... */ });
    if (response.ok) {
      const { token, user } = await response.json();
      localStorage.setItem('token', token); // Persist session
      setUser(user);
      setToken(token);
    }
  };
};
```

```

return (
  <AuthContext.Provider value={{ user, token, login }}>
    {children}
  </AuthContext.Provider>
);
};

```

Chapter 6: Asset Management & Cloudinary

6.1 Media Uploads (cloudinary.js)

We never store images on our own server; we stream them to Cloudinary.

```

const storage = new CloudinaryStorage({
  cloudinary: cloudinary,
  params: async (req, file) => {
    const folders = getFolderPaths(req.user.email); // Dynamic folder per user
    return {
      folder: folders.assets, // Organization: users/email/assets
      allowed_formats: ['jpg', 'png', 'pdf'],
      resource_type: 'auto' // Detects if it's an image or a PDF document
    };
  }
});

```

6.2 Data Integrity Sync (assetUpdater.js)

When an agent changes their phone number or logo, it must reflect on all their listings instantly.

```

const updateUserAssetsAgent = async (userId, updateData) => {
  const models = [CarAsset, BikeAsset, YachtAsset, EstateAsset, Listing];

  // Build update query dynamically
  const updateQuery = {};
  if (updateData.name) updateQuery['agent.name'] = updateData.name;
  if (updateData.photo) updateQuery['agent.photo'] = updateData.photo;

  // Update thousands of documents across multiple collections in parallel
  const updatePromises = models.map(Model =>
    Model.updateMany({ 'agent.id': userId.toString() }, { $set: updateQuery })
  );
  await Promise.all(updatePromises);
};

```

Chapter 7: Search Engine Optimization (SEO)

7.1 Dynamic Metadata (SEO.jsx)

```

export default function SEO({ title, description, image }) {
  const seoTitle = title ? `${title} | Otulia` : 'Otulia - Luxury Assets';

  return (
    <Helmet>
      <title>{seoTitle}</title> // Dynamic browser tab title
      <meta name="description" content={description} /> // Search engine snippet
      <meta property="og:image" content={image} /> // Image for Facebook/iMessage
    </Helmet>
  );
}

```

7.2 Dynamic Sitemap Logic (sitemap.routes.js)

```

router.get('/sitemap.xml', async (req, res) => {
  // 1. Fetch live data from DB
  const [cars, yachts] = await Promise.all([
    CarAsset.find({ status: 'Active' }).select('_id updatedAt'),
    YachtAsset.find({ status: 'Active' }).select('_id updatedAt')
  ]);

  // 2. Generate XML string on-the-fly
  let xml = `<?xml version="1.0" encoding="UTF-8"?><urlset xmlns="...">`;
  cars.forEach(c => {
    xml += `<url><loc>https://otulia.com/asset/car/${c._id}</loc></url>`;
  });

  // 3. Serve as XML
  res.header('Content-Type', 'application/xml');
  res.send(xml);
});

```